

Parsers Ascendentes

Edward Hermann Haeusler

Informática - PUC-Rio - Brasil

INF1022 - Análise Léxica e Sintática
Maio 2022

Análise Ascendente (Shift-Reduce)

1. **Shift** $(\alpha, ay) \xrightarrow{\text{shift}} (\alpha a, y)$. Aplica-se **shift** quando não existe redução possível e a entrada é válida (como saber?).
2. **Reduce** $(\alpha\beta, y) \xrightarrow{A \rightarrow \beta} (\alpha A, y)$.
3. **Error** $(\alpha, ay) \xrightarrow{\text{error}} ?$. Como saber quando ay não é válido?

$$\begin{aligned} S &\longrightarrow E \\ E &\longrightarrow T \mid T + E \\ T &\longrightarrow id \mid (E) \end{aligned}$$

Vantagens sobre parsers descendentes?

*Parsers $LL(k)$ devem ser capazes de prever que **produção** usar com base somente em k símbolos a direita. No caso de $LL(1)$ a partir de somente um símbolo.*

*Um parser $LR(k)$ pode adiar a decisão de que produção seguir até ler todo o lado direito de uma regra, e, ainda se utilizar de k símbolos a frente (**lookahead**).*

Parser LR(0): Não usa nenhum símbolo a direita na predição

Uso de duas tabelas:

1. Tabela de ações: $Act[estado, simbolo]$
2. Tabela de gotos: $Goto[estado, simbolo]$

Construção das tabelas usa o conceito de ítem e de configuração:

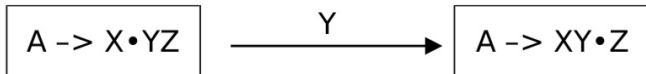
- ▶ Ítems derivados da regra $A \rightarrow XYZ$: $A \rightarrow \bullet XYZ$, $A \rightarrow X \bullet YZ$, $A \rightarrow XY \bullet Z$ e $A \rightarrow XYZ \bullet$.
- ▶ Configuração é um conjunto de ítems correspondentes a um mesmo estágio de análise sintática: Se $A \rightarrow X \bullet YZ$ está em uma configuração e $Y \rightarrow \omega_1$ e $Y \rightarrow \omega_2$ são regras da gramática então $Y \rightarrow \bullet \omega_1$ e $Y \rightarrow \bullet \omega_2$ devem estar nesta mesma configuração também. Chama-se a isso de fechamento (closure).

A gramática aumentada

Introdução de uma nova regra inicial com símbolo E' dá conta de adequar a pilha para aceitação. O estado associado a $E' \rightarrow E \bullet \$$ é adicionado às configurações e é inicialmente empilhado.

- ▶ (0) $E' \longrightarrow E\$$
- ▶ (1) $E \longrightarrow T$
- ▶ (2) $E \longrightarrow E + T$
- ▶ (3) $T \longrightarrow id$
- ▶ (4) $T \longrightarrow (E)$

Transição entre itens



Configurações e tabela sucessores (MEF)

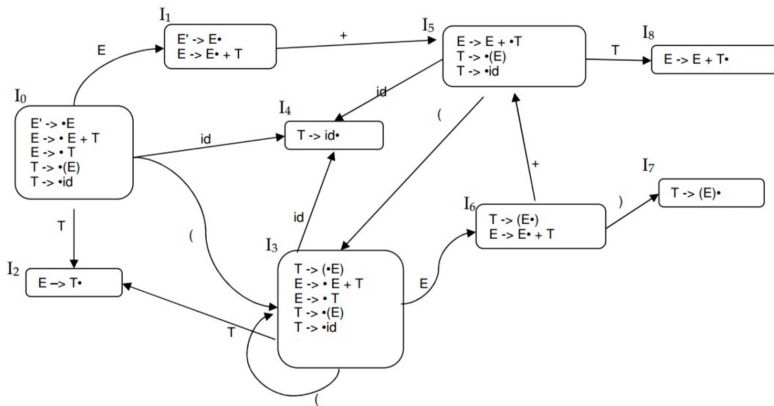


Tabela de ações e gotos: Construção

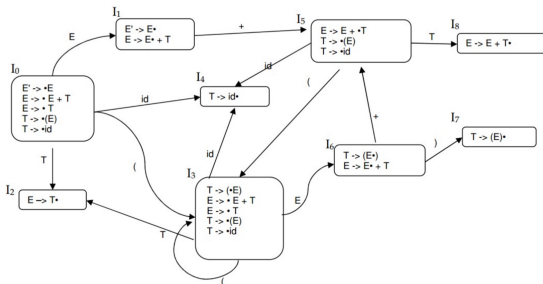
1. Construa (defina) as configurações e os sucessores a partir da gramática G ;
2. Considere os estados $\{0, 1, \dots, k\}$ como as configurações $\{l_0, \dots, l_k\}$. As ações são definidas como:
 - 2.1 Se $A \rightarrow \omega \bullet$ está em l_i então $Act[i, \sigma] = reduce(y)$, onde y é o número da regra $A \rightarrow \omega$, para todo $\sigma \in \Sigma_G$, exceto quando $A = S'$;
 - 2.2 Se $S' \rightarrow S \bullet$ está em l_i então $Act[i, \$] = accept$;
 - 2.3 Se $A \rightarrow \omega_1 \bullet \sigma \omega_2$ está em l_i e $sucessor[i, \sigma] = j$ então $Act[i, \sigma] = shift(j)$, com $\sigma \in \Sigma_G$.
3. $GoTo[i, A] = sucessor[i, A]$, para todo i e $A \in N_G$, i.e., A não terminal da gramática;
4. Todas as entradas não definidas tem valor *error*;
5. O estado inicial l_0 é o que contém $S' \rightarrow \bullet S$.

LR(0), SLR, LR(k) e LaLR: O algoritmo

Algorithm 1 Parser Asc sobre as tabelas Action and Goto

```
1:  $stack \leftarrow I_0; token \leftarrow ProxToken()$ 
2: repeat
3:    $s \leftarrow TOP(stack)$ 
4:   if  $action[s, token] = shift_i$  then
5:      $PUSH(shift_i); token \leftarrow ProxToken()$ 
6:   else if  $action[s, token] = reduce(A \rightarrow \gamma)$  then
7:      $POP \mid \gamma \mid \text{symbols}$ 
8:      $s \leftarrow TOP(stack)$ 
9:      $PUSH(goto[s, A])$ 
10:  else if  $action[s, token] = accept$  then
11:     $return(accept)$ 
12:  else
13:     $return(error)$ 
14:  end if
15: until  $error \text{ OR } accept$ 
```

Passo a Passo



Algorithm 1 Parser Asc sobre as tabelas Action and Goto

```

1:  $stack \leftarrow I_0; token \leftarrow ProxToken()$ 
2: repeat
3:    $s \leftarrow TOP(stack)$ 
4:   if  $action[s, token] = shift_i$  then
5:      $PUSH(shift_i); token \leftarrow ProxToken()$ 
6:   else if  $action[s, token] = reduce(A \rightarrow \gamma)$  then
7:      $POP \mid \gamma \mid symbols$ 
8:      $s \leftarrow TOP(stack)$ 
9:      $PUSH(goto[s, A])$ 
10:  else if  $action[s, token] = accept$  then
11:     $return(accept)$ 
12:  else
13:     $return(error)$ 
14:  end if
15: until  $error$  OR  $accept$ 
    
```

Importante

Nos métodos $LR(k)$, $SLR(k)$ e $LaLR(k)$ não é necessário o empilhamento dos símbolos da gramática (terminais e não-terminais). Os estados já estão associados a símbolos terminais e/ou não-terminais. O empilhamento de estados é suficiente nestes métodos de análise sintática.

No exemplo, o estado I_4 indica implicitamente que um *id* foi lido e empilhado, I_5 que um '+' foi lido e empilhado, etc. Já o empilhamento implícito de não-terminais nem chega a ser executado, dado que na linha 9 do algoritmo, o que é empilhado é o sucessor do estado que representa o empilhamento. Trata-se de uma pequena otimização local.

Obs: O JFLAP, que é uma ferramenta de apoio ao ensino, implementa os métodos ascendentes com empilhamento tanto de símbolos terminais quanto de não-terminais para facilitar o entendimento do aluno que só conhece PDAs teóricos.



Exemplo de parsing LR(0)

STATE STACK	REMAINING INPUT	PARSER ACTION
S ₀	id + (id)\$	Shift S ₄ onto state stack, move ahead in input
S ₀ S ₄	+ (id)\$	Reduce 4) T → id, pop state stack, goto S ₂ , input unchanged
S ₀ S ₂	+ (id)\$	Reduce 2) E → T, goto S ₁
S ₀ S ₁	+ (id)\$	Shift S ₅
S ₀ S ₁ S ₅	(id)\$	Shift S ₃
S ₀ S ₁ S ₅ S ₃	id)\$	Shift S ₄
S ₀ S ₁ S ₅ S ₃ S ₄	)\$	Reduce 4) T → id, goto S ₂
S ₀ S ₁ S ₅ S ₃ S ₂	)\$	Reduce 2) E → T, goto S ₆
S ₀ S ₁ S ₅ S ₃ S ₆	)\$	Shift S ₇
S ₀ S ₁ S ₅ S ₃ S ₆ S ₇	\$	Reduce 3) T → (E), goto S ₈
S ₀ S ₁ S ₅ S ₈	\$	Reduce 1) E → E + T, goto S ₁
S ₀ S ₁	\$	Accept

Obs: Os estados $I_0, \dots, I_8$ estão representados por $S_0, \dots, S_8$

Limitações de gramáticas $LR(0)$

Não pode haver conflitos **shift-reduce** nem **reduce-reduce**, i.e.:

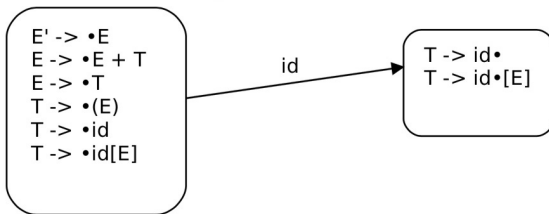
Em cada I_i não pode existir um ítem da forma $A \rightarrow \omega_1 \bullet \sigma \omega_2$ e outro na forma $B \rightarrow \omega_3 \bullet$ (conflito **shift-reduce**)

Em cada I_i não pode existir mais de um ítem na forma $B \rightarrow \omega_3 \bullet$ (conflito **reduce-reduce**). A tabela de sucessor deve ser determinística.

Corolário: Gramáticas com regras ϵ não são $LR(0)$ em geral.

Melhorando $LR(0)$

- ▶ (0) $E' \longrightarrow E$
- ▶ (1) $E \longrightarrow T$
- ▶ (2) $E \longrightarrow E + T$
- ▶ (3) $T \longrightarrow id$
- ▶ (4) $T \longrightarrow (E)$
- ▶ (5) $T \longrightarrow id[E]$

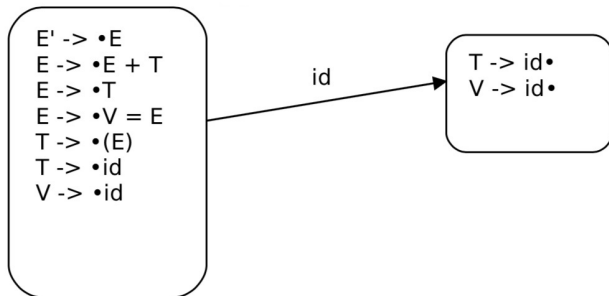


A construção da tabela SLR(1)

1. Construa (defina) as configurações e os sucessores a partir da gramática G ;
2. Considere os estados $\{0, 1, \dots, k\}$ como as configurações $\{l_0, \dots, l_k\}$. As ações são definidas como:
 - 2.1 Se $A \rightarrow \omega \bullet$ está em l_i então $Act[i, \sigma] = reduce(y)$, onde y é o número da regra $A \rightarrow \omega$, **para todo** $\sigma \in Follow(A)$, exceto quando $A = S'$;
 - 2.2 Se $S' \rightarrow S \bullet$ está em l_i então $Act[i, \$] = accept$;
 - 2.3 Se $A \rightarrow \omega_1 \bullet \sigma \omega_2$ está em l_i e $sucessor[i, \sigma] = j$ então $Act[i, \sigma] = shift(j)$, com $\sigma \in \Sigma_G$.
3. $GoTo[i, A] = sucessor[i, A]$, para todo i e $A \in N_G$, i.e., A não terminal da gramática;
4. Todas as entradas não definidas tem valor *error*;
5. O estado inicial l_0 é o que contém $S' \rightarrow \bullet S$.

Outra gramática que é SLR ($SLR(1)$) e não é $LR(0)$

- ▶ (0) $E' \rightarrow E$
- ▶ (1) $E \rightarrow T$
- ▶ (2) $E \rightarrow E + T$
- ▶ (3) $E \rightarrow V = E$
- ▶ (4) $T \rightarrow (E)$
- ▶ (5) $T \rightarrow id$
- ▶ (6) $V \rightarrow id$



Limitações de $SLR(1)$

A análise $SLR(1)$ não usa toda a informação disponível na pilha para evitar conflitos do tipo **shift-reduce**

Conflito **shift-reduce** não eliminável com **FOLLOW**

- ▶ (0) $S' \longrightarrow S\$$
- ▶ (1) $S \longrightarrow L = R$
- ▶ (2) $S \longrightarrow R$
- ▶ (3) $L \longrightarrow *R$
- ▶ (4) $L \longrightarrow id$
- ▶ (5) $R \longrightarrow L$

Obs: Gramática [Aho,Ullman,Sethi] para atribuição (assignment) com “l-value”, “r-value” e “contents-of” (*).

Construindo as configurações...

$I_0:$ $S' \rightarrow \bullet S \$$
 $S \rightarrow \bullet L = R$
 $S \rightarrow \bullet R$
 $L \rightarrow \bullet *R$
 $L \rightarrow \bullet id$
 $R \rightarrow \bullet L$

$I_5:$ $L \rightarrow id \bullet$

$I_6:$ $S \rightarrow L = \bullet R$
 $R \rightarrow \bullet L$
 $L \rightarrow \bullet *R$
 $L \rightarrow \bullet id$

$I_1:$ $S' \rightarrow S \bullet \$$

$I_7:$ $L \rightarrow *R \bullet$

$I_2:$ $S \rightarrow L \bullet = R$
 $R \rightarrow L \bullet$

$I_8:$ $R \rightarrow L \bullet$

$I_9:$ $S \rightarrow L = R \bullet$

$I_3:$ $S \rightarrow R \bullet$

$I_4:$ $L \rightarrow * \bullet R$
 $R \rightarrow \bullet L$
 $L \rightarrow \bullet *R$
 $L \rightarrow \bullet id$

Atenção! $\Rightarrow$ Conflito shift-reduce no estado I_2 .

Analisando a palavra (comando na LP) $id = id$ (I)

I_0 : $S' \rightarrow \bullet S \$$
 $S \rightarrow \bullet L = R$
 $S \rightarrow \bullet R$
 $L \rightarrow \bullet * R$
 $L \rightarrow \bullet id$
 $R \rightarrow \bullet L$

I_5 : $L \rightarrow id \bullet$

I_6 : $S \rightarrow L = \bullet R$
 $R \rightarrow \bullet L$
 $L \rightarrow \bullet * R$
 $L \rightarrow \bullet id$

I_1 : $S' \rightarrow S \bullet \$$

I_7 : $L \rightarrow * R \bullet$

I_2 : $S \rightarrow L \bullet = R$
 $R \rightarrow L \bullet$

I_8 : $R \rightarrow L \bullet$

I_3 : $S \rightarrow R \bullet$

I_9 : $S \rightarrow L = R \bullet$

I_4 : $L \rightarrow * \bullet R$
 $R \rightarrow \bullet L$
 $L \rightarrow \bullet * R$
 $L \rightarrow \bullet id$

$(I_0, id = id) \xrightarrow{\text{shift5}} (I_0 I_5, = id) \xrightarrow{\text{reduce2}} (I_0 I_2, = id)$

Como “=” $\in \text{Follow}(R)$, $\text{Act}[2, =]$ é shift6 ou reduce ?

$\Rightarrow$ O problema não está na gramática !!!

Analizando $id = id$ (II)

reduce não deve ser o caso, pois $R = \dots$ não é um prefixo válido.

O conteúdo da pilha indica que o **shift6** é a ação correta!

$I_0:$ $S' \rightarrow \bullet S \$$
 $S \rightarrow \bullet L = R$
 $S \rightarrow \bullet R$
 $L \rightarrow \bullet * R$
 $L \rightarrow \bullet id$
 $R \rightarrow \bullet L$

$I_5:$ $L \rightarrow id \bullet$

$I_6:$ $S \rightarrow L = \bullet R$
 $R \rightarrow \bullet L$
 $L \rightarrow \bullet * R$
 $L \rightarrow \bullet id$

$I_1:$ $S' \rightarrow S \bullet \$$

$I_7:$ $L \rightarrow * R \bullet$

$I_2:$ $S \rightarrow L \bullet = R$
 $R \rightarrow L \bullet$

$I_8:$ $R \rightarrow L \bullet$

$I_3:$ $S \rightarrow R \bullet$

$I_9:$ $S \rightarrow L = R \bullet$

$I_4:$ $L \rightarrow * \bullet R$
 $R \rightarrow \bullet L$
 $L \rightarrow \bullet * R$
 $L \rightarrow \bullet id$

Análise LR(1)

Diferenciando as derivações

$$S \Rightarrow L = R$$

$$S \Rightarrow R \Rightarrow L$$

Mesmo com “=” em $Follow(R)$, ele não deve vir na sequência do R na segunda derivação acima. Somente o \$ deve seguir o R nesta segunda derivação.

Esta informação está na Pilha (I_0 é o estado abaixo do topo)

Construção de tabelas $LR(1)$

Estados com **lookahead**: $(A \rightarrow B_1 \dots B_k \bullet B_{k+1} \dots B_j, a)$

corresponde a: (1) A pilha tem $B_1 \dots B_k$; (2) procura por $B_{k+1} \dots B_j$ na pilha e reduz (3) somente se o token lido após o B_j é “a”.

Os estados relevantes na análise $LR(1)$ são da forma $(A \rightarrow B_1 \dots B_k \bullet, a)$, indicando uma redução da forma $A \rightarrow B_1 \dots B_k$ somente se o **lookahead** é “a”, i.e., se o token após B_k é a . Usa-se $(A \rightarrow B_1 \dots B_k \bullet, a/b/c)$ quando o **lookahead** pode ser qualquer um dentre a , b ou c .

Nova forma de calcular o closure

Repete até que nenhuma adição seja feita a configuração I :

Se $(A \rightarrow \omega_1 \bullet B\omega_2, a) \in I$ então:

*Para cada $B \rightarrow \omega \in G$, e, para todo $b \in \text{First}(\omega_2 a)$
inclua $(B \rightarrow \omega, b)$ em I , caso já não pertença a I .*

O closure é mais preciso que em $SLR(1)$: Somente são consideradas as produções de B com **lookahead** na sequência da continuação dos itens da configuração. O **lookahead** induz uma partição nos estados.

Cálculo do sucessor

Se $(A \rightarrow \omega_1 \bullet B\omega_2, a) \in I$ então

$Sucessor(I, B) = \text{closure}((A \rightarrow \omega_1 B \bullet \omega_2, a))$

O cálculo das configurações e da tabela de sucessores começa com $I_0 = \text{closure}((S' \rightarrow \bullet S, \$))$

Um exemplo (Aho,Ullman, Sethi, pp,231-236)

- ▶ (0) $S' \longrightarrow S\$$
- ▶ (1) $S \longrightarrow XX$
- ▶ (2) $X \longrightarrow aX$
- ▶ (3) $X \longrightarrow b$

$I_0:$ $S' \rightarrow \bullet S, \$$
 $S \rightarrow \bullet XX, \$$
 $X \rightarrow \bullet aX, a/b$
 $X \rightarrow \bullet b, a/b$

$I_1:$ $S' \rightarrow S \bullet, \$$

$I_2:$ $S \rightarrow X \bullet X, \$$
 $X \rightarrow \bullet aX, \$$
 $X \rightarrow \bullet b, \$$

$I_3:$ $X \rightarrow a \bullet X, a/b$
 $X \rightarrow \bullet aX, a/b$
 $X \rightarrow \bullet b, a/b$

$I_4:$ $X \rightarrow b \bullet, a/b$

$I_5:$ $S \rightarrow XX \bullet, \$$

$I_6:$ $X \rightarrow a \bullet X, \$$
 $X \rightarrow \bullet aX, \$$
 $X \rightarrow \bullet b, \$$

$I_7:$ $X \rightarrow b \bullet, \$$

$I_8:$ $X \rightarrow aX \bullet, a/b$

$I_9:$ $X \rightarrow aX \bullet, \$$

A construção das tabelas $LR(1)$

1. Construa (defina) as configurações e os sucessores a partir da gramática G ;
2. Considere os estados $\{0, 1, \dots, k\}$ como as configurações $\{l_0, \dots, l_k\}$. As ações são definidas como:
 - 2.1 Se $(A \rightarrow \omega \bullet, a)$ está em l_i então $Act[i, a] = reduce(y)$, onde y é o número da regra $A \rightarrow \omega$, exceto quando $A = S'$;
 - 2.2 Se $(S' \rightarrow S \bullet, \$)$ está em l_i então $Act[i, \$] = accept$;
 - 2.3 Se $(A \rightarrow \omega_1 \bullet a \omega_2, b)$ está em l_i e $sucessor[i, a] = j$ então $Act[i, a] = shift(j)$, com $a \in \Sigma_G$.
3. $GoTo[i, A] = sucessor[i, A]$, para todo i e $A \in N_G$, i.e., A não terminal da gramática;
4. Todas as entradas não definidas tem valor *error*;
5. O estado inicial l_0 é o que contém $(S' \rightarrow \bullet S, \$)$.

As tabelas LR(1)

I_0 :	$S' \rightarrow \bullet S, \$$ $S \rightarrow \bullet XX, \$$ $X \rightarrow \bullet aX, a/b$ $X \rightarrow \bullet b, a/b$	I_4 :	$X \rightarrow b\bullet, a/b$
I_1 :	$S' \rightarrow S\bullet, \$$	I_5 :	$S \rightarrow XX\bullet, \$$
I_2 :	$S \rightarrow X\bullet X, \$$ $X \rightarrow \bullet aX, \$$ $X \rightarrow \bullet b, \$$	I_6 :	$X \rightarrow a\bullet X, \$$ $X \rightarrow \bullet aX, \$$ $X \rightarrow \bullet b, \$$
I_3 :	$X \rightarrow a\bullet X, a/b$ $X \rightarrow \bullet aX, a/b$ $X \rightarrow \bullet b, a/b$	I_7 :	$X \rightarrow b\bullet, \$$
		I_8 :	$X \rightarrow aX\bullet, a/b$
		I_9 :	$X \rightarrow aX\bullet, \$$

Figura: Estados

State on top of stack	Action			Goto	
	a	b	\$	S	X
0	s3	s4		1	2
1			acc		
2	s6	s7			5
3	s3	s4			8
4	r3	r3			
5			r1		
6	s6	s7			9
7			r3		
8	r2	r2			
9			r2		

Figura: Tabela *Action* e *Goto*

Analizando *baab*\$

State on top of stack	Action			Goto	
	a	b	\$	S	X
0	s3	s4		1	2
1			acc		
2	s6	s7			5
3	s3	s4			8
4	r3	r3			
5			r1		
6	s6	s7			9
7			r3		
8	r2	r2			
9			r2		

Figura: Estados

STACK STACK	REMAINING INPUT	PARSER ACTION
S ₀	baab\$	Shift S ₄
S ₀ S ₄	aab\$	Reduce 3) X -> b, goto S ₂
S ₀ S ₂	aab\$	Shift S ₆
S ₀ S ₂ S ₆	ab\$	Shift S ₆
S ₀ S ₂ S ₆ S ₆	b\$	Shift S ₇
S ₀ S ₂ S ₆ S ₆ S ₇	\$	Reduce 3) X -> b, goto S ₉
S ₀ S ₂ S ₆ S ₆ S ₉	\$	Reduce 2) X -> aX, goto S ₉
S ₀ S ₂ S ₆ S ₉	\$	Reduce 2) X -> aX, goto S ₅
S ₀ S ₂ S ₅	\$	Reduce 1) S -> XX, goto S ₁
S ₀ S ₁	\$	Accept

Figura: Tabela *Action* e *Goto*

Analísadores LaLR(1)

LR(1) é um método bem geral e poderia ser sempre usado, **porém**, o tamanho da **MEF** é muito grande. LPs como *C++* ou Java tem tabelas LR(1) com mais de 3000 estados.

Gramáticas SLR(1) para estas LPs¹ tem em torno de 300 estados.

LaLR(1) é proposta como uma solução intermediária, usa **lookaheads** como em LR(1), mas obtém-se muito menos estados. Obs: É um método bastante popularizado pelo **Yacc** (Yet Another Compiler Compilers).

¹Com a incorporação de algumas restrições

O kernel do LaLR(1)

O **kernel** de um conjunto de itens LR(1) $\mathcal{S}$ é o conjunto de itens LR(0) obtido de $\mathcal{S}$ pela retirada dos **lookaheads**

A tabela LaLR(1) colapsa todos os estados LR(1) com o mesmo **kernel** em um mesmo estado LaLR(1).

A MEF LaLR(1)

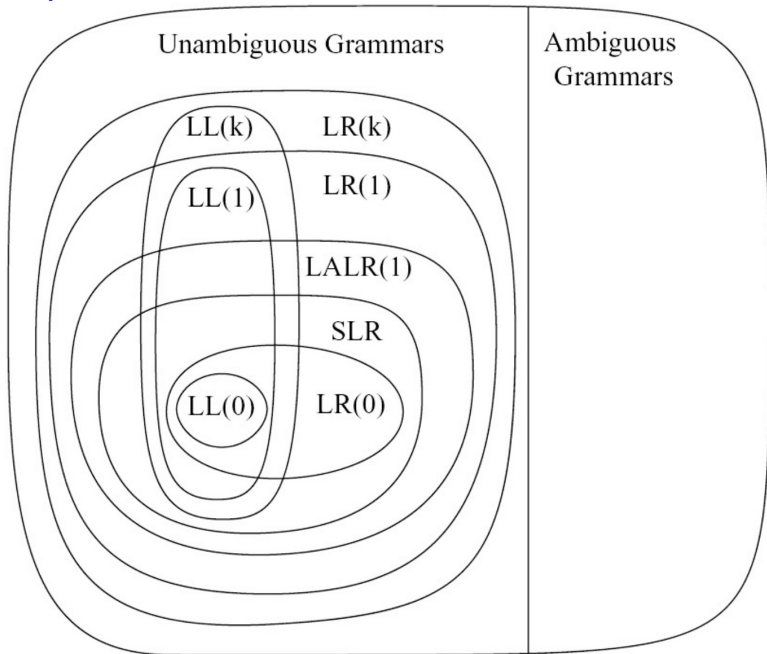
A partir de um parser LR(1):

- ▶ Identifique estados LR(1) com mesmo kernel: Para conjunto de estados LR(1) $\{S_1, \dots, S_n\}$ com mesmo kernel construa um estado LaLR(1) $S = \bigcup_{i=1,n} S_i$;
- ▶ $Sucessor_{LaLR(1)}(S, V) = S'$, tal que, $Sucessor_{LR(1)}(S_i, V) = S'_i$ e $S'_i \in S'$;

Tabela de ações: Se o item $B \rightarrow \omega \bullet$, a está em S então

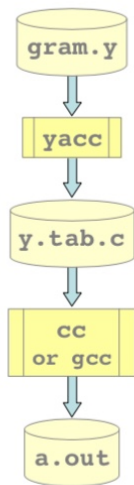
$Act[S, a] = reduce(B \rightarrow \omega)$

Hierarquia de LLC



YACC

Seqüência básica operacional



Arquivo contendo gramática desejada no formato yacc

programa yacc

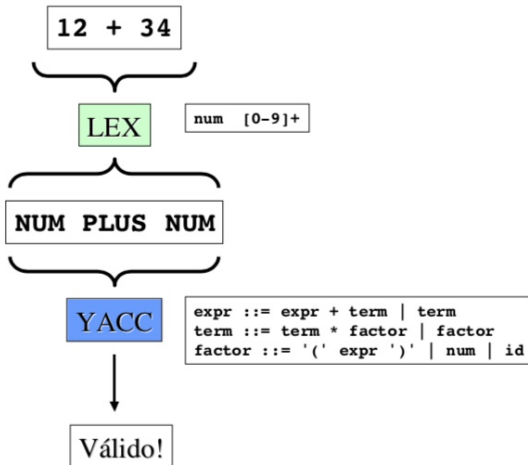
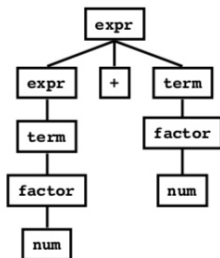
programa fonte C criado pelo yacc

Compilador C

Programa executável que faz a análise sintática da gramática descrita em parse.y

Exemplo bem simples (prof. Sandro Rigo (UNICAMP))

Exemplo



Formato do arquivo YACC

Definições

%%

Regras

%%

Código Suplementar

Seções de Regras

- Normalmente escritas como segue:

```
expr      : expr '+' term  
          | term  
          ;  
  
term      : term '*' factor  
          | factor  
          ;  
  
factor    : '(' expr ')'  
          | ID  
          | NUM  
          ;
```

Seção de Definições

```
%{  
#include <stdio.h>  
#include <stdlib.h>  
%}  
  
%token ID NUM  
  
%start expr
```

Obs:

- lex produz uma função **yylex()**
- yacc produz uma função **yyparse()**
- yyparse espera chamar uma yylex
- **Como conseguir yylex?**
 - Escrever sua própria!
 - Usar Lex

Formato do arquivo YACC

Definições

%%

Regras

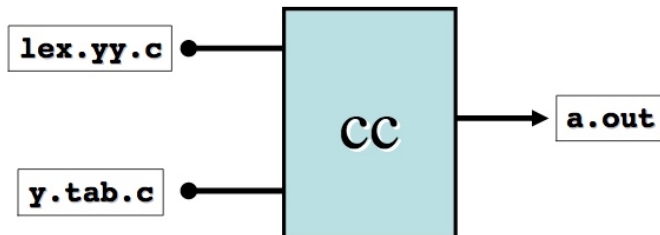
%%

Código Suplementar

Seu próprio yylex()

```
int yylex()
{
    if(it's a num)
        return NUM;
    else if(it's an id)
        return ID;
    else if(parsing is done)
        return 0;
    else if(it's an error)
        return -1;
```

lex & yacc



Exemplo

- Suponha um arquivo lex **scanner.l** e um arquivo yacc chamado **decl.y**.
- Passos a serem feitos ...

```
yacc -d decl.y  
lex scanner.l  
gcc -c lex.yy.c y.tab.c  
gcc -o parser lex.yy.o y.tab.o -ll
```

Nota: scanner deve incluir na seção de definições:

```
#include "y.tab.h"
```

YACC

- As regras podem ser recursivas
- As regras não podem ser ambíguas*
- Usa um parser bottom up Shift/Reduce -LALR(1)
 - Solicita um token
 - Empilha
 - Redução ?
 - Sim: reduz usando a regras correspondente
 - Não: pega outro token
- Yacc não pode olhar mais que um token de lookahead
- `yacc -v gram.y` gera a tabela de estados, em `y.output`

Exemplo Yacc Simples

OBRIGADO !!!